

1 Lesson 02-10-26

1.1 Control hazards

We now move on to discussing **control hazards**. We explicitly ignored them until now as they are:

- Slightly more complicated;
- Open up a world of possibilities for CPU optimizations.

They are due to branches (conditional and unconditional), which may change the PC after the following instruction has been fetched already. In the case of conditional branches, the decision on whether the PC should be modified (branch taken) or not (branch untaken) can be taken even later. In the basic RISC-V implementation, the PC is written with the target address (if the jump is taken) at the end of the EXE stage, i.e., 2 clock cycles after the IF of the branch. This behavior delays the target instruction by two clock cycles.

1.1.1 Stalling

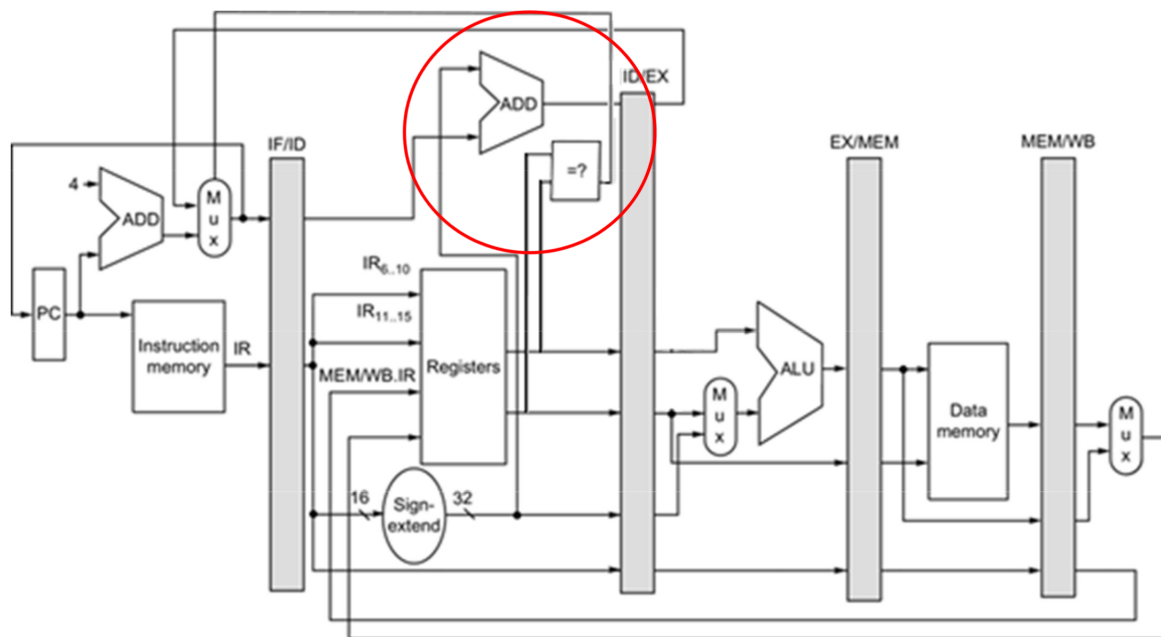
A possible solution is based on stalling the pipeline as soon as a branch instruction is detected (ID stage) by:

- Deciding earlier whether the branch has to be taken or not;
- Computing earlier the new PC value.

This means that we make a temporary decision and continue loading the pipeline as if the branch wasn't there (or as if it was always taken, depending on the prediction). If the branch does, in fact, get taken (or not taken) then we wasted clock cycles and have to flush the pipeline. Otherwise, we win.

1.1.2 Stalling with modified pipeline

This clearly has an advantage in modifying the pipeline for early branch address computation (in the ID stage instead of the EX stage).



Note that this approach has an issue: at most we can put the branch prediction in the ID (decode) stage, that is the second stage of the branch instruction processing. This means that the instruction loaded in the pipeline right after the branch one is always wasted (never depends on the branch prediction).

We show a figure that details this situation.

Branch instruction	IF	ID	EX	MEM	WB		
Branch +1		IF	--	--	--	--	
Branch Target			IF	ID	EX	MEM	WB
Branch Target + 1				IF	ID	EX	MEM

From this, we introduce the fact that several techniques can be used to manage branches:

- Freezing the pipeline;
- Predict untaken;
- Predict taken;
- Delayed branch.

1.1.3 Freezing the pipeline

This is the previously proposed solution: the pipeline is stalled (or flushed) as soon as a branch instruction is detected, and until the decision about the branch is known. It is the simplest solution to implement.

It wastes 2 clock cycles (without the modified pipeline), or at least 1 (with the modified pipeline, decode is second stage).

1.1.4 Predict untaken

This technique introduces a *guess*. The *predictions* we talked about so far weren't predictions in the strict sense: at most we were making the actual condition calculation earlier (in the ID stage), but we still were sure about actual branches taken. Now we assume an arbitrary value of the condition before evaluating it:

- We assume the branch is not taken;
- We avoid any change in the pipeline status until the branch decision has been taken;
- We undo all the performed operations if the branch turns out to be taken.

Considering our last solution, this makes the most sense as we loaded the instruction after the branch anyways: if the branch isn't taken, there's no point in flushing.

Untaken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB
Taken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	idle	idle	idle	idle			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

1.1.5 Predict taken

The same technique can be done by *guessing* the branch is taken. The results are the same, just that we load the instruction after the target (instead of the one after the branch). Performance gains are similar assuming branches get taken or not taken with equal probability.

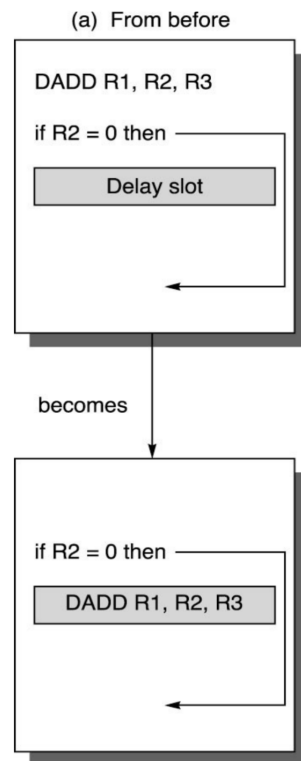
Note the role of the *compiler*: If the hardware supports the predict taken or predict untaken scheme, the compiler can improve performance by generating code which maximizes the chance for the processor to make the right prediction.

For instance: considering the loop implementation, the for scheme is suitable for the predict untaken scheme, the do while for the predict taken. In fact, the for scheme involves executing a conditional branch instruction before the loop body, which results in a not taken branch as many time as the number of iterations. the do while does the opposite.

1.1.6 Delayed branch

This technique is based on filling the slot after the branch instruction (named branch-delay slot) with instructions which have to be executed no matter the branch outcome. It is an actual *architectural* change on the ISA: branches actually branch only after one instruction, and the slot which was wasted before now has a fixed purpose.

It is up to the compiler (or the seasoned programmer) to fill each branch-delay slot with the right instructions. The processor does nothing special when a branch instruction is decoded.



MIPS famously used one branch delay slot. ARM also historically had behavior resembling this in older ARM architectures, where instructions following a branch could already have entered the pipeline, but ARM's architectural rules are different from the classic MIPS "one guaranteed delay-slot instruction" model. Today, other techniques are preferred.

The performance depends on the compiler ability in finding the right instructions to put in the delay slots. We find that using this technique, only about 30% of branches do produce a penalty.

When deeply pipelined processors are used, the delay slots is longer, and the advantages of delayed-branches smaller. Therefore, such RISC architectures do not support delayed branches.

1.2 Multicycle operations

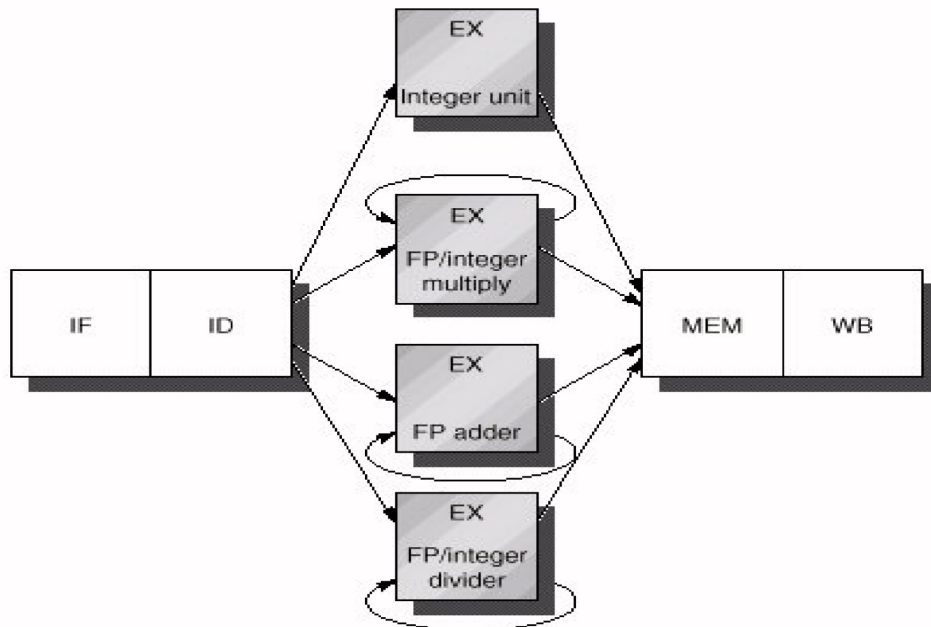
Let's talk *complex operations*, such as **floating point**. Floating point units perform more complex operations than integer ones. Therefore, in order to force them to perform their job in a single clock cycle, the designer should:

- Either use a very slow clock;
- Or make these units very complex.

As a popular alternative, floating point units generally require more than one clock cycle to complete. The EX stage is composed of different functional units, and takes as many clock cycles, as the instruction requires.

1.2.1 Multicycle execution

The resulting pipeline is as follows, with the EX stage divided in multiple units with feedback and (possibly) multicycle instruction permanence.



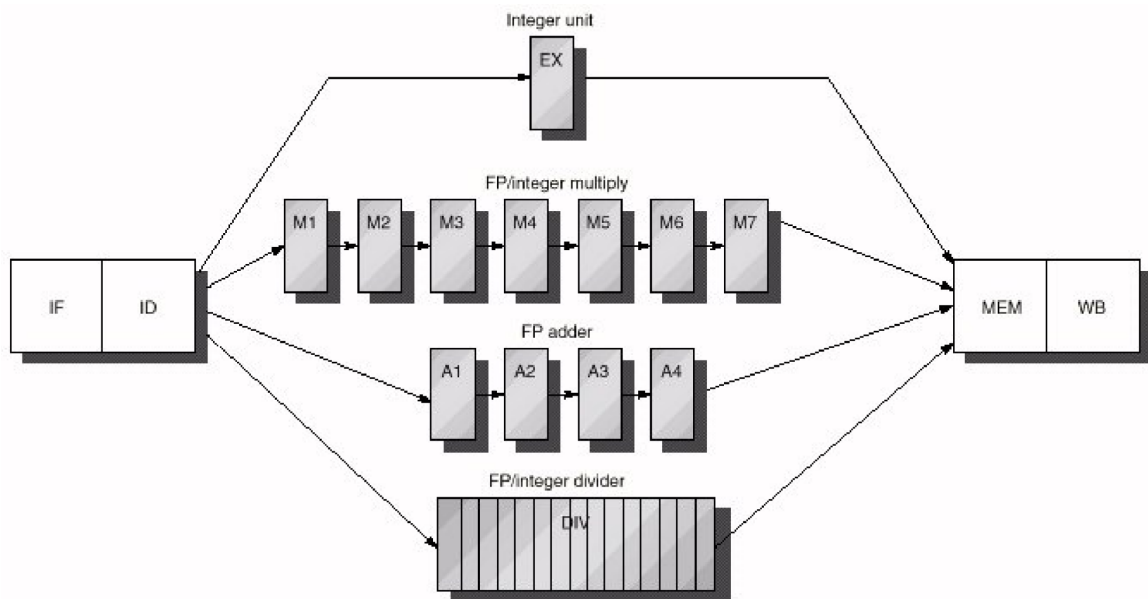
We see the problems of this approach by considering certain *metrics* of the execution phase:

- **Latency:** it is the number of cycles that should last between an instruction that produces a result and an instruction that uses the same result;
- **Initiation interval:** it is the number of cycles that must elapse between issuing two operations of the same type to the same unit.

This approach increases latency as some instructions stay in the execution stage longer.

1.2.2 Multicycle pipelines

An improvement is to *pipeline* our expanded execution stage:



Each instruction, as with all pipelines, stays in the processor the same amount of time (if not longer because of pipelining overhead), but at least we have the increased throughput from more instruction coexisting in the pipeline at the same time.

One thing to note, though, is that due to the different structure of the EX stage, *hazards* may become more frequent.

1.2.3 Multicycle hazards

Let's see the **hazards** we can encounter with this architecture.

- **Structural hazards** can occur in ways we didn't encounter before:
 - Because of the unpipelined divide unit (at least in our example above), several instructions could need it at the same time;
 - Because the instructions have varying running times, the number of register writes required in a cycle can be larger than 1 (and this applies generally).

Instruction	Clock cycle number										
	1	2	3	4	5	6	7	8	9	10	11
fmul.d f0, f4, f6	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
fadd.d f2, f4, f6				IF	ID	A1	A2	A3	A4	MEM	WB
...					IF	ID	EX	MEM	WB		
...						IF	ID	EX	MEM	WB	
fld f2, 0(x2)							IF	ID	EX	MEM	WB

This is particularly bad as in our integer pipeline there was no way to have WB stages overlap.

The solutions, as always (we saw structural hazards on the integer pipeline), are:

- Adding other write ports (normally too expensive)
- Forcing a structural hazard via *stalling*:
 - * Instructions are stalled in the ID stage;
 - * Or instructions are stalled before entering the MEM or WB stage.
- **Data hazards** can occur, are more frequent and have an higher performance impact.
 - Because of longer latency of operations, stalls for data hazards may stall the pipeline for longer periods.

Instruction	Clock cycle number																
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
f1d f4,0(x2)	IF	ID	EX	MEM	WB												
fmul.d f0,f4,f6		IF	ID	Stall	M1	M2	M3	M4	M5	M6	M7	MEM	WB				
fadd.d f2,f0,f8			IF	Stall	ID	Stall	Stall	Stall	Stall	Stall	Stall	A1	A2	A3	A4	MEM	WB
f2d f2,0(x2)					IF	Stall	Stall	Stall	Stall	Stall	Stall	ID	EX	Stall	Stall	Stall	MEM

As with the before example, the long stall is due to the hefty execution size of the floating point instructions, which keeps the target register "locked" for more than our integer pipeline did.

This is a case of a **RAW** (*Read After Write*) hazard;

- The concurrent WB retires are also worth noting: new kinds of data hazards might manifest because multiple instructions reach WB at the same time.

This is a case of a **WAW** (*Write After Write*) hazard.

The simple solution is: before issuing an instruction to the EXE stage, check whether it is going to write on the same register of an instruction still in the EXE stage. In this case, stall the new instruction.

1.2.4 Summary of hazards

To summarize, if hazard detection is always performed in the ID stage, three checks have to be performed:

- Structural hazards (involving the divide unit and the write port);
- RAW data hazards: check whether some source register is listed among the destination registers of pending instructions, and whether this register will not be available at the right moment
- WAW data hazards: check whether the instruction currently in ID has the same destination register of any instruction in EXE.